

ArgoMAC security proof blueprint

Kriterion

September 23, 2026

Chapter 1

Introduction

This blueprint describes the three verified security properties of the submitted ArgoMAC garbling scheme for BN254 scalar multiplication from the BABE paper [1]. The scheme garbles the function $f[r](\pi)$ of [1, Eq. 27]: it returns $r\pi$ for a point $\pi \in \mathbb{G}_1$ and 0 otherwise, and it hides the secret scalar r . The Lean project extracts each node from the source with LeanArchitect. Each node shows the Lean declaration, its statement in the notation of the paper, and its dependencies. The dependency graph on the web version shows the proof tree of each property. The roots match the `perfectCorrectness`, `adaptivePrivacy`, and `lambportCompatible` fields of `Submission.solution`.

1.1 Paper correspondence

Each node names its counterpart in the BaBe paper [1]. The references use BABE v2.0, the version pinned by the Lean sources (`commit e2dcf4d`). Each chapter restates the items of the BaBe paper before the Lean nodes, with the kind of the item in the paper: theorem, lemma, claim, definition, construction, or algorithm. Each restatement adapts the wording of the paper to the notation of this blueprint and names the item, for example “Lemma 14 of the BaBe paper”. Table 1.1 lists the paper items and the Lean nodes that formalize them.

Paper item	Lean nodes
Fig. 21, overall garbling scheme	theorems 9, 10, 12, 13, 16 to 18 and 22
Eq. 27, $f[r]$ rejects off-curve inputs	theorems 14 and 15
Fig. 17 and Fig. 18, circuit C_4	theorem 19
Claim 7 and Section 5.5, Enc_t through EncPRF	theorem 20
Fig. 9 and Fig. 10, circuit C_2	theorems 21, 23 and 24
Eqs. 29–31, Jacobian coordinates of $r_j\pi + K_j$	theorems 25 to 29
Fig. 6 and Fig. 7, circuit C_1	theorems 30, 31 and 34
Appendix C and Alg. 8, base- $(2-\omega)$ digits	theorems 32 and 33
Theorem 8, adaptive privacy of the optimized scheme	theorems 45 and 46
Theorem 7, hybrid Hyb_0	theorem 47
Lemma 14, concrete bound for GC_3 in the pRPM	theorems 48 to 50, 59, 62, 65 and 66
Lemma 13, H-coefficient technique	theorems 51 and 52
Claim 11, bad transcript mass ε_1	theorems 53, 60 and 61
Claim 12, good transcript ratio	theorems 54 to 56
Definition 20, bad transcripts	theorems 57 and 58
Construction 3, simulator $(\text{Sim}_1, \text{Sim}_2)$	theorems 63, 64 and 67 to 75
Eqs. 15–16 and Section 5.1, Lamport labels	theorems 11 and 78 to 87

Table 1.1: Paper items and the Lean nodes that formalize them.

Chapter 2

Perfect correctness

The root is theorem 9. For every secret scalar r , every random tape, and every input π , Eval on the garbled circuit and the labels of π returns $f[r](\pi)$ (Fig. 21). The proof follows the evaluation chain of the paper: the curve membership circuit C_4 releases t , Dec_t recovers the labels L_3 , the garbled C_2 returns the Jacobian coordinates of $r_j\pi + K_j$, and Eval_1 sums them with the base- $(2 - \omega)$ weights. Figure 2.1 shows the proof tree. An arrow points from a lemma to the result that uses it.

2.1 Statements from the paper

Definition 1 (Correctness; Definition 3 of the BaBe paper [1]). A garbling scheme $\text{GC} = (\text{Garble}, \text{Encode}, \text{Eval})$ is correct if for every circuit C and every input x :

$$\Pr [C(x) = \text{Eval}(\text{ct}_{\text{GC}}, L_x) : (\text{ct}_{\text{GC}}, \text{ek}) \leftarrow \text{Garble}(C), L_x \leftarrow \text{Encode}(\text{ek}, x)] = 1.$$

Formalized by theorem 9.

Definition 2 (The function $f[r]$; Eq. 27 of the BaBe paper [1]). The Verifier fixes a secret scalar $r \in \mathbb{F}_r^*$. The Prover supplies a public input point π through the bit decomposition $\mathbf{b} = (x_0(\pi), \dots, x_{n-1}(\pi), y_0(\pi), \dots, y_{n-1}(\pi))$ of its affine coordinates, with $n = 254$. The garbled function is

$$f[r](\pi) = \begin{cases} r\pi & \text{if } \pi \in \mathbb{G}_1, \\ 0 & \text{otherwise.} \end{cases}$$

Formalized by theorem 13; the off-curve branch is theorem 14.

Construction 3 (Overall garbling scheme; Fig. 21 and Fig. 29 of the BaBe paper [1]). $\text{Garble}(C[r_0, \dots, r_{\ell-1}])$ runs $\text{Garble}_1(C_1)$, $\text{Garble}_2(C_2[\text{ek}_1])$, $\text{Garble}_3(C_3[\text{ek}_2])$, samples $t, w \leftarrow \mathbb{F}_p$, runs $\text{Garble}_4(C_4[t, w])$ and $\text{Garble}_5(C_5[\text{ek}_4])$, and returns $\text{ct}_{\text{gc}} = (\text{ct}_{\text{gc},2}, \text{ct}_{\text{gc},3}, \text{ct}_{\text{gc},4}, \text{ct}_{\text{gc},5})$ and $\text{ek} = (\text{ek}_3, t, \text{ek}_5)$. $\text{Encode}(\text{ek}, \mathbf{b})$ returns $L = (\text{Enc}_t(L_3), L_5)$ with $L_3 = \text{Encode}_3(\text{ek}_3, \mathbf{b})$ and $L_5 = \text{Encode}_5(\text{ek}_5, \mathbf{b})$. $\text{Eval}(\text{ct}_{\text{gc}}, L, \mathbf{b})$ computes $L_4 \leftarrow \text{Eval}_5$, $\hat{t} \leftarrow \text{Eval}_4(\text{ct}_{\text{gc},4}, L_4, x(\pi), y(\pi))$, $L_3 \leftarrow \text{Dec}_{\hat{t}}(\text{Enc}_t(L_3))$, $L_2 \leftarrow \text{Eval}_3$, the Jacobian coordinates $(X, Y, Z)(r_j\pi + K_j)_j \leftarrow \text{Eval}_2(\text{ct}_{\text{gc},2}, L_2, x(\pi), y(\pi))$, converts them to affine points $L_{1,j}$, and returns $\text{Eval}_1(\perp, L_1)$. The optimized scheme (Fig. 29) uses the base- $(2 - \omega)$ digits $r_j \in D$ with $\ell = 92$ and merges the layers into GC_{23} and GC_{45} . Formalized by theorems 16 to 18 and 22.

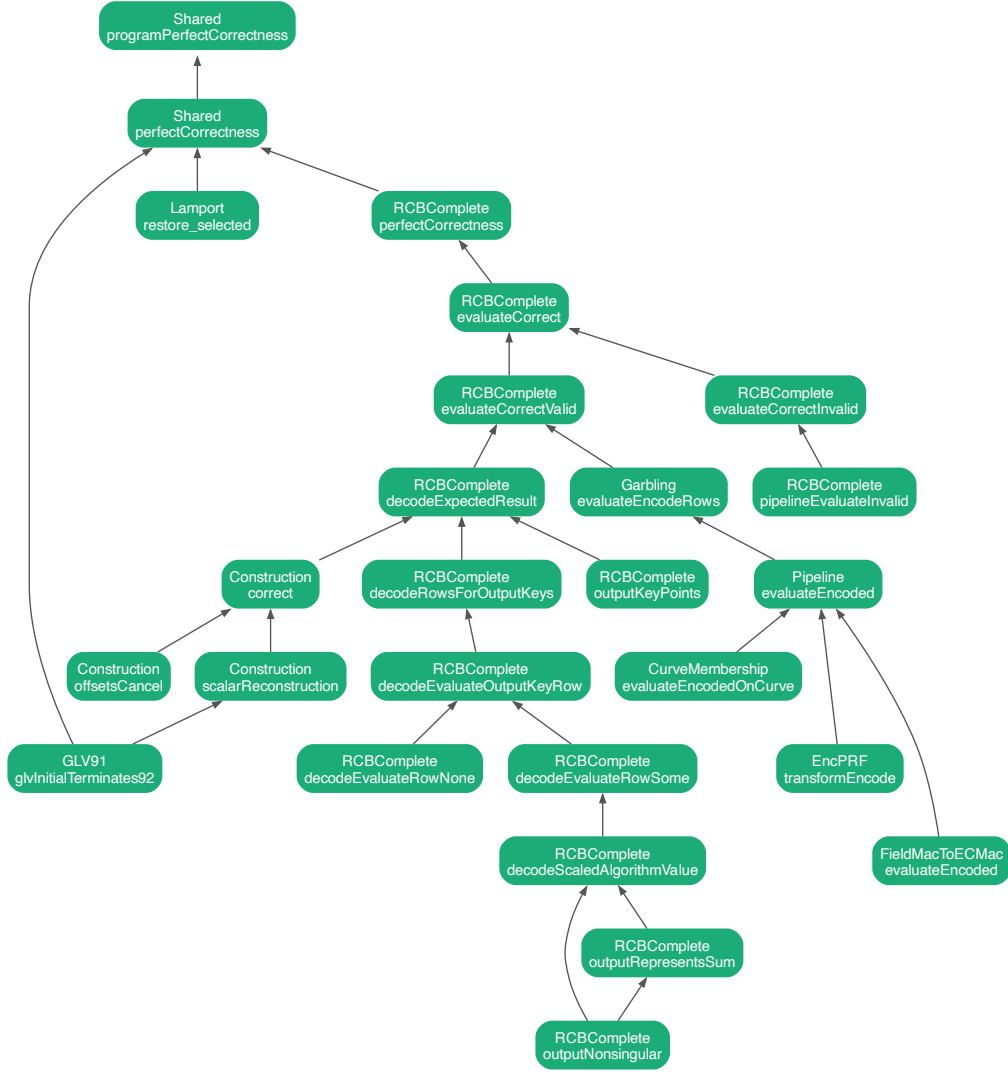


Figure 2.1: Dependencies of the perfect correctness proof.

Construction 4 (Circuit C_1 and its garbling; Fig. 6 and Fig. 7 of the BaBe paper [1]). $C_1[r_0, \dots, r_{\ell-1}](\pi) = \sum_{j=0}^{\ell-1} (2 - \omega)^j (r_j \pi)$. Garble_1 samples $K_j \leftarrow \mathbb{G}_1$ for $j \geq 1$, sets $K_0 = -\sum_{j \geq 1} (2 - \omega)^j K_j$, and returns $\text{ct}_{\text{gc},1} = \perp$ and $\text{ek}_1 = (r_0, \dots, r_{\ell-1}, K_0, \dots, K_{\ell-1})$. $\text{Encode}_1(\text{ek}_1, \pi)$ returns $L_1 = (L_{1,j})_j$ with $L_{1,j} = r_j \pi + K_j$. $\text{Eval}_1(\perp, L_1) = \sum_{j=0}^{\ell-1} (2 - \omega)^j L_{1,j}$. Formalized by theorems 30, 31 and 34.

Construction 5 (Circuit C_2 and its garbling; Fig. 9, Fig. 10, Eqs. 29–31 of the BaBe paper [1]). For each digit j and coordinate $W \in \{X, Y, Z\}$, the Jacobian coordinate $W(r_j \pi + K_j)$ is a quadratic function of $(x(\pi), y(\pi))$:

$$W(r_j \pi + K_j) = c_0^{(j,W)} + c_1^{(j,W)} x(\pi) + c_2^{(j,W)} y(\pi) + c_3^{(j,W)} x(\pi)y(\pi) + c_4^{(j,W)} x(\pi)^2 + c_5^{(j,W)} y(\pi)^2.$$

For a binary digit r_j , an offset $K_j = (x(K_j), y(K_j))$, a randomizer $a_j \leftarrow \mathbb{F}_p$, and $B = 3$, the coefficients are

$$\begin{aligned} \mathbf{c}^{(j,X)} &= a_j^2 (r_j \cdot 2B + (1 - r_j)x(K_j), r_j x(K_j)^2, -2r_j y(K_j), 0, r_j x(K_j), 0), \\ \mathbf{c}^{(j,Y)} &= a_j^3 (r_j \cdot 3B y(K_j) + (1 - r_j)y(K_j), 0, r_j(-y(K_j)^2 - 3B), -3r_j x(K_j)^2, 3r_j x(K_j)y(K_j), r_j y(K_j)), \\ \mathbf{c}^{(j,Z)} &= a_j (-r_j x(K_j) + (1 - r_j), r_j, 0, 0, 0, 0). \end{aligned}$$

The base- $(2 - \omega)$ implementation uses the adjusted coefficients of [1, Eqs. 51–53] for digits $r_j \in D$. Garble_2 applies the Ishai–Wee partial garbling: it samples masks $\rho_1^{(j,W)}, \dots, \rho_{10}^{(j,W)}$, sets $\rho_0 = -\rho_9 - \rho_{10}$, and publishes $c'_k = c_k + \rho_k$ for $k \leq 5$. Encode_2 returns the labels $c'_6, \dots, c'_{10}$, which are affine in $x(\pi)$ and $y(\pi)$. Eval_2 returns $c'_0 + c'_1 x + c'_2 y + c'_3 xy + c'_4 x^2 + c'_5 y^2 + c'_6 x + c'_7 y + c'_8 y + c'_9 + c'_{10} = W(r_j \pi + K_j)$. Formalized by theorems 21 and 23 to 29.

Construction 6 (Curve membership circuit C_4 ; Fig. 17 and Fig. 18 of the BaBe paper [1]). $C_4[t, w](x, y) = t + w(x^3 + 3 - y^2) = (t + 3w) + w x^3 - w y^2$ for $t, w \leftarrow \mathbb{F}_p$. The value t is released if and only if π is on the curve. Garble_4 applies the Ishai–Wee technique to the coefficients $(\hat{c}_0, \hat{c}_1, \hat{c}_2) = (t + 3w, w, -w)$ with masks $\hat{\rho}_1, \dots, \hat{\rho}_7$, and Eval_4 returns $\tilde{c}'_0 + \tilde{c}'_1 x^3 + \tilde{c}'_2 y^2 + \tilde{c}'_3 x^2 + \tilde{c}'_4 y + \tilde{c}'_5 x + \tilde{c}'_6 + \tilde{c}'_7$. Formalized by theorem 19.

Claim 7 (Encryption of the labels; Claim 7 and Eq. 33 of the BaBe paper [1]). *(Enc, Dec) is implemented with a PRF EncPRF and a public counter: $\text{Enc}_t(m) = \text{EncPRF}(t, \text{counter}) \oplus m$ and $\text{Dec}_t(\text{ct}) = \text{EncPRF}(t, \text{counter}) \oplus \text{ct}$. $\text{EncPRF}(t, (w, i, b)) = \text{P}_{w,i}(\text{enc}(b) \oplus k_1) \oplus k_2$ with $k_1 \| k_2 = \text{SHA256}(t)$ is the two-key Even–Mansour construction, where each $\text{P}_{w,i}$ is a programmable random permutation. The labels of GC_3 are encrypted label by label: $\hat{L}_{x,i}^b = \text{Enc}_t(L_{x,i}^b)$ and $\hat{L}_{y,i}^b = \text{Enc}_t(L_{y,i}^b)$ (Eq. 33). If GC_{45} is ϵ_1 -adaptively private, then the first two hybrids of the privacy proof differ by at most $\epsilon_1 + \text{negl}(\lambda)$. Formalized by theorem 20.*

Algorithm 8 (Base- $(2 - \omega)$ digits; Appendix C and Alg. 8 of the BaBe paper [1]). Let $\omega \in \mathbb{F}_r$ with $\omega^2 + \omega + 1 = 0$ and $D = \{0, \pm 1, \pm \omega, \pm(1 + \omega)\}$. For every scalar $r \in \{0, \dots, r - 1\}$, the GLV decomposition gives small integers (a, b) with $r \equiv a + \omega b \pmod{r}$. Alg. 8 divides $A + B\omega$ by $2 - \omega$ repeatedly, with the digit $r_i \in D$ chosen from $(A + 2B) \bmod 7$ and the exact division $A' = (3A - B)/7$, $B' = (A + 2B)/7$, and stops at $(A, B) = (0, 0)$ within $\ell = 1 + \lceil \log_7 r \rceil = 92$ iterations. The reconstruction $\sum_{i=0}^{\ell-1} r_i (2 - \omega)^i \equiv r \pmod{r}$ holds. Multiplication by a digit in D is an endomorphism: $\omega \pi = (\mu x(\pi), y(\pi))$ with $\mu^2 + \mu + 1 = 0$ in $\mathbb{F}_p$, and $-\pi = (x(\pi), -y(\pi))$. Formalized by theorems 32 and 33.

2.2 Lean statements

2.2.1 Program and wire circuits

Theorem 9 (BABE Fig. 21). *Lean: Kriterion.ArgoMAC.Shared.programPerfectCorrectness*

Perfect correctness of the garbling scheme (Garble, Encode, Eval) of Fig. 21 for $f[r]$ (Eq. 27). For every secret scalar r , every random tape, and every input $\pi = (x(\pi), y(\pi))$: $\text{Eval}(\text{ct}_{\text{gc}}, \text{Encode}(\text{ek}, \pi), \pi) = f[r](\pi)$, where $(\text{ct}_{\text{gc}}, \text{ek}) = \text{Garble}(C[r])$. The tape supplies the private coins and the public oracles CTPRF, EncPRF, and the hash.

Proof. Fix r , the tape, and π . Apply theorem 10 to the tape whose public oracle is replaced by the oracle component of the tape. Unfold the program circuit, the wire circuit, and the label map. The two sides agree. $\square$

Theorem 10 (BABE Fig. 21). *Lean: Kriterion.ArgoMAC.Shared.perfectCorrectness*

Perfect correctness of the wire circuit, the garbling scheme of Fig. 21 whose input labels are the Lamport signatures of the bits of π (Eqs. 15–16). For every coherent tape and every input π , evaluation under the evaluation oracle returns $f[r](\pi)$.

Proof. Fix r , a coherent tape, and π . Unfold the wire circuit and rewrite the evaluation oracle with the tape property. By theorem 11 the Lamport signature restores the bit labels L . Conclude with theorem 12. $\square$

Theorem 11 (BABE Eqs. 15–16). *Lean: Kriterion.ArgoMAC.Lamport.restore_selected*

The wire adapter inverts the Lamport encoding. For every input π and every label vector $L = \{L_{x,i}^{x_i(\pi)}, L_{y,i}^{y_i(\pi)}\}_{i < 254}$, restoring L from its 508 Lamport signature slots returns the bits of π together with L .

Proof. Compare the x and y label vectors index by index. For $i < 254$ slot i holds $L_{x,i}^{x_i(\pi)}$. For $i \geq 254$ slot i holds $L_{y,i-254}^{y_{i-254}(\pi)}$. $\square$

Theorem 12 (BABE Fig. 21). *Lean: Kriterion.ArgoMAC.RCBCComplete.perfectCorrectness*

Perfect correctness of the garbled circuit of Fig. 21 for the fixed base- $(2 - \omega)$ construction with $\ell = 92$ digits. The random tape supplies the fixed-key permutations of CTPRF, the permutations of EncPRF, and the hash oracle.

Proof. Fix r , the randomness, and π . The claim is theorem 13 wrapped in some. $\square$

Theorem 13 (BABE Fig. 21). *Lean: Kriterion.ArgoMAC.RCBCComplete.evaluateCorrect*

For every nonzero scalar r , every randomness, and every input π : $\text{Eval}(\text{ct}_{\text{gc}}, \text{Encode}(\text{ek}, \pi), \pi) = f[r](\pi)$, where $(\text{ct}_{\text{gc}}, \text{ek}) = \text{Garble}(C[r])$ (Fig. 21).

Proof. Split on whether π decodes to a point of $\mathbb{G}_1$. The off-curve case is theorem 14. The on-curve case is theorem 16. $\square$

2.2.2 Invalid inputs

Theorem 14 (BABE Eq. 27). *Lean: Kriterion.ArgoMAC.RCBCComplete.evaluateCorrectInvalid*

When π is not a point of $\mathbb{G}_1$, Eval returns $f[r](\pi) = 0$ (Eq. 27). The curve membership check $C_4[t, w]$ (Fig. 17) does not release t , so the labels L_3 stay encrypted.

Proof. Unfold Eval. By theorem 15 the evaluation chain returns $\perp$. The function $f[r]$ also returns 0 for an off-curve input. $\square$

Theorem 15 (BABE Eq. 27). *Lean: Kriterion.ArgoMAC.RCBCComplete.pipelineEvaluateInvalid*
When π is not a point of $\mathbb{G}_1$, the evaluation chain $\text{Eval}_5 \rightarrow \text{Eval}_4 \rightarrow \text{Dec}_t \rightarrow \text{Eval}_3 \rightarrow \text{Eval}_2$ of Fig. 21 returns $\perp$ for every ct_{gc} and every label vector L .

Proof. Unfold the chain. The first step decodes π as a curve point and fails, so the chain returns $\perp$. $\square$

2.2.3 Valid inputs

Theorem 16 (BABE Fig. 21). *Lean: Kriterion.ArgoMAC.RCBCComplete.evaluateCorrectValid*
When π decodes to a point $P \in \mathbb{G}_1$: $\text{Eval}(\text{ct}_{\text{gc}}, \text{Encode}(\text{ek}, \pi), \pi) = f[r](\pi) = rP$.

Proof. Unfold Eval into the evaluation chain followed by the decoder. By theorem 17 the chain returns the Jacobian coordinates of $r_j\pi + K_j$ for all j . The offsets K_j are clamped, so theorem 22 decodes these coordinates to rP . $\square$

Theorem 17 (BABE Fig. 21). *Lean: Kriterion.ArgoMAC.Garbling.evaluateEncodeRows*
For an on-curve input π and the encoding key ek of Garble, the evaluation chain of Fig. 21 applied to ct_{gc} and $\text{Encode}(\text{ek}, \pi)$ returns the Jacobian coordinates $(X, Y, Z)(r_j\pi + K_j)$ of Eval_2 for all $j \in \{0, \dots, \ell - 1\}$.

Proof. This is theorem 18 applied to the output keys (r_j, K_j) , the randomizers a_j , and the oracles of ek . $\square$

Theorem 18 (BABE Fig. 21). *Lean: Kriterion.ArgoMAC.Pipeline.evaluateEncoded*
Correctness of the evaluation chain. For an on-curve input π , evaluating the garbled table on the bit labels $L = \text{Encode}(\text{ek}, \pi)$ returns $\text{Eval}_2(\text{ct}_{\text{gc}, 2}, L_2, x(\pi), y(\pi))$, the Jacobian coordinates of $r_j\pi + K_j$ for all j (Fig. 10).

Proof. Since π decodes to a point, π is on the curve. Unfold Eval and Garble. By theorem 19 the garbled $C_4[t, w]$ releases $\hat{t} = t$. By theorem 20 decrypting the encrypted labels $\text{Enc}_t(L_3)$ with t gives the labels L_3 of π . By theorem 21 the garbled C_2 evaluates to the expected coordinates, because its rows are sparse. $\square$

Theorem 19 (BABE Fig. 18). *Lean: Kriterion.ArgoMAC.CurveMembership.evaluateEncodedOnCurve*
Correctness of the garbled $C_4[t, w]$ (Fig. 18). For all $t, w, r_1, r_2 \in \mathbb{F}_p$, every input π on the curve $y^2 = x^3 + 3$, and the labels of π : Eval_4 returns $t + w(x(\pi)^3 + 3 - y(\pi)^2) = t$.

Proof. Rewrite the evaluation on the labels of π . The curve equation gives $x(\pi)^3 + 3 - y(\pi)^2 = 0$, which removes the masked term. The remaining terms cancel by ring arithmetic. $\square$

Theorem 20 (BABE Claim 7). *Lean: Kriterion.ArgoMAC.EncPRF.transformEncode*
Encrypting labels commutes with selecting labels. For the permutation oracle of EncPRF , whitening keys, an encoding key ek_3 , and input bits $\mathbf{b}$: $\text{Enc}_t(\text{Encode}_3(\text{ek}_3, \mathbf{b})) = \text{Encode}_3(\text{Enc}_t(\text{ek}_3), \mathbf{b})$, where $\text{Enc}_t(\text{ek}_3)$ encrypts every label as in Eq. 33 and Claim 7.

Proof. Compare the x and y label vectors. Each coordinate follows from the transformation of one coordinate encoding. $\square$

Theorem 21 (BABE Fig. 10). *Lean: Kriterion.ArgoMAC.FieldMacToECMac.evaluateEncoded*
Correctness of the garbled C_2 (Fig. 10). For garbled rows $\{c_k'^{(j, W)}\}$, randomizers a_j , oracles, an encoding key, and an input π : if every row is sparse, then Eval_2 on the labels of π returns the expected result $\{W(r_j\pi + K_j)\}_{j, W}$ of the rows.

Proof. Unfold Eval_2 . The evaluation of the homogeneous labels equals the expected result, because every row is sparse. $\square$

2.2.4 Decoding

Theorem 22 (BABE Fig. 21). *Lean: Kriterion.ArgoMAC.RCBCComplete.decodeExpectedResult*
Decoding the output of Eval_2 gives rP . For clamped offsets K_j , randomizers a_j , and an input π that decodes to P : converting the Jacobian coordinates $(X, Y, Z)(r_j\pi + K_j)$ to affine coordinates and running Eval_1 (Fig. 21) returns rP .

Proof. Unfold the decoder and the expected result. By theorem 23 the rows decode to the points $r_jP + K_j$. By theorem 30 this list equals $L_1 = (L_{1,0}, \dots, L_{1,\ell-1})$ of Encode_1 . By theorem 31 $\text{Eval}_1(\perp, L_1) = rP$. $\square$

Theorem 23 (BABE Fig. 9). *Lean: Kriterion.ArgoMAC.RCBCComplete.decodeRowsForOutputKeys*
For output keys $(r_j, K_j)_j$, randomizers a_j , and an input π that decodes to P : converting every evaluated row of C_2 from Jacobian to affine coordinates returns the list of points $r_jP + K_j$ (Fig. 9).

Proof. Unfold the point decoder and the rows. Decode each row with theorem 24. The vector of successful decodings is the expected list. $\square$

Theorem 24 (BABE Fig. 9). *Lean: Kriterion.ArgoMAC.RCBCComplete.decodeEvaluateOutputKeyRow*
One row of C_2 decodes to $r_jP + K_j$. For an output key (r_j, K_j) with digit $r_j \in D = \{0, \pm 1, \pm \omega, \pm(1 + \omega)\}$, a nonzero randomizer a_j , and an input π that decodes to P : converting the evaluated row (X, Y, Z) to affine coordinates returns $r_jP + K_j$.

Proof. Since π decodes to a point, π is on the curve. Split on the endomorphism base of the digit r_j . If $r_j = 0$, theorem 25 returns K_j . Otherwise the base is a sixth root of unity μ with $r_j\pi = (\mu^k x(\pi), \pm y(\pi))$, and theorem 26 returns $K_j + r_jP$. $\square$

Theorem 25 (BABE Eqs. 29–31). *Lean: Kriterion.ArgoMAC.RCBCComplete.decodeEvaluateRowNone*
For the digit $r_j = 0$, an offset K_j , and a nonzero randomizer a_j : the row of C_2 evaluates to a Jacobian representation of K_j and decodes to K_j .

Proof. Rewrite the row for the zero digit. The decoder divides by the nonzero randomizer and recovers K_j in affine coordinates. $\square$

Theorem 26 (BABE Eqs. 29–31). *Lean: Kriterion.ArgoMAC.RCBCComplete.decodeEvaluateRowSome*
For a digit $r_j \neq 0$ with endomorphism base μ , $\mu^6 = 1$, an offset K_j , a nonzero randomizer a_j , and an on-curve input π : the row of C_2 decodes to $K_j + \pi'$, where $\pi' = r_j\pi = (\mu^k x(\pi), \pm y(\pi))$ is the transformed input.

Proof. The transformed input π' stays on the curve because $\mu^6 = 1$. Rewrite the row as the Jacobian addition of K_j and π' scaled by a_j . Conclude with theorem 27. $\square$

Theorem 27 (BABE Eqs. 29–31). *Lean: Kriterion.ArgoMAC.RCBCComplete.decodeScaledAlgorithmValue*
Jacobian addition (Eqs. 29–31) decodes to the affine sum. For two on-curve points K and π and a nonzero randomizer a : converting the scaled output $(a^2 X, a^3 Y, aZ)$ of the addition formula to affine coordinates returns $K + \pi$.

Proof. By theorem 28 the output of the addition formula is a nonsingular projective point. Scaling by a keeps it nonsingular. The decoder returns the affine form of this point, and scaling does not change the affine form. By theorem 29 the affine form is $K + \pi$. $\square$

Theorem 28 (BABE Eqs. 29–31). *Lean: Kriterion.ArgoMAC.RCBCComplete.outputNonsingular*
For two on-curve points (a, b) and (u, v) , the output (X, Y, Z) of the addition formula of Eqs. 29–31 is a nonsingular point of the projective BN254 curve.

Proof. The output satisfies the projective curve equation and is not the zero vector. If $Z = 0$, then $X = 0$ and $Y \neq 0$, so the point is the nonsingular point at infinity. If $Z \neq 0$, the affine point satisfies the curve equation, and the nonzero discriminant gives nonsingularity. $\square$

Theorem 29 (BABE Eqs. 29–31). *Lean: Kriterion.ArgoMAC.RCBCComplete.outputRepresentsSum*
For two on-curve points (a, b) and (u, v) , the affine form of the output of the addition formula equals the group sum $(a, b) + (u, v)$ in $\mathbb{G}_1$.

Proof. Split on whether $a \neq u$. If they differ, $Z \neq 0$, and the recovered coordinates match the affine addition formula. If $a = u$, the y coordinates are equal or opposite. Equal y coordinates give the doubling formula with $Z \neq 0$. Opposite y coordinates give $Z = 0$, the point at infinity, and the affine sum is also 0. $\square$

Theorem 30 (BABE Fig. 7). *Lean: Kriterion.ArgoMAC.RCBCComplete.outputKeyPoints*
The list of points $r_j P + K_j$ over all output keys equals $L_1 = \text{Encode}_1(\text{ek}_1, P)$ of Fig. 7, where the offsets are clamped by $K_0 = -\sum_{j \geq 1} (2 - \omega)^j K_j$.

Proof. Read the output keys as the digit vector $(r_0, \dots, r_{\ell-1})$ paired with the offsets. Rewrite the vector as a zip of the digit multiples with the offset points. The clamped successful offsets are $(K_0, \dots, K_{\ell-1})$. The zip is the definition of Encode_1 . $\square$

2.2.5 Randomized encoding

Theorem 31 (BABE Fig. 6). *Lean: Kriterion.ArgoMAC.Construction.correct*
Correctness of C_1 (Fig. 6): $\text{Eval}_1(\perp, L_1) = \sum_{j=0}^{\ell-1} (2 - \omega)^j (r_j P + K_j) = rP$ for every scalar r , every offset randomness, and every point P .

Proof. Unfold Encode_1 . The Horner sum splits into $(\sum_j r_j (2 - \omega)^j)P$ plus $\sum_j (2 - \omega)^j K_j$. By theorem 32 the first factor is r . By theorem 34 the second sum is 0. $\square$

Theorem 32 (BABE Alg. 8). *Lean: Kriterion.ArgoMAC.Construction.scalarReconstruction*
Reconstruction of Alg. 8: $\sum_{j=0}^{\ell-1} r_j (2 - \omega)^j \equiv r \pmod{\mathfrak{r}}$ for the base- $(2 - \omega)$ digits $r_j \in D$ of every scalar r , with $\ell = 92$. The termination certificate uses theorem 33.

Proof. The reconstruction invariant holds for 92 iterations of Alg. 8 with the remaining pair (A, B) as a tail. By the termination certificate $(A, B) = (0, 0)$ after 92 iterations, so the tail vanishes. The initial pair (a, b) satisfies $a + \omega b \equiv r$ by the GLV decomposition. $\square$

Theorem 33 (BABE Appendix C). *Lean: Kriterion.ArgoMAC.GLV91.glvInitialTerminates92*
Alg. 8 terminates. For every scalar r , the GLV pair (a, b) with $r \equiv a + \omega b \pmod{\mathfrak{r}}$ reaches $(A, B) = (0, 0)$ after 92 iterations of the digit loop (Appendix C).

Proof. It is enough that the norm of (a, b) is at most the bound for 91 iterations. The GLV coefficients satisfy the bounds of the decomposition. Bound a^2 , b^2 , and $-ab$ by the basis constants. A numeric check compares the constants with the norm bound. $\square$

Theorem 34 (BABE Fig. 7). *Lean: Kriterion.ArgoMAC.Construction.offsetsCancel*
The clamped offsets of Garble_1 (Fig. 7) cancel: $\sum_{j=0}^{\ell-1} (2 - \omega)^j K_j = 0$ for every offset randomness, because $K_0 = -\sum_{j \geq 1} (2 - \omega)^j K_j$.

Proof. The clamp lemma cancels the Horner sum of the free offsets $K_1, \dots, K_{\ell-1}$. $\square$

Chapter 3

Adaptive privacy

The root is theorem [45](#). The requirement is the work-per-advantage level of Lemma 14: every adversary with work T has advantage δ with $T/\delta \geq 2^{100}$. The proof follows the H-coefficient technique of Lemma 13 in the programmable random permutation model, with the bad transcripts of Definition 20, the bad-transcript mass ε_1 of Claim 11, and the good-transcript ratio of Claim 12. The simulator $(\text{Sim}_1, \text{Sim}_2)$ of Construction 3 is compiled to a bounded machine. Figure [3.1](#) shows the proof tree. Grey nodes belong to the perfect correctness chapter.

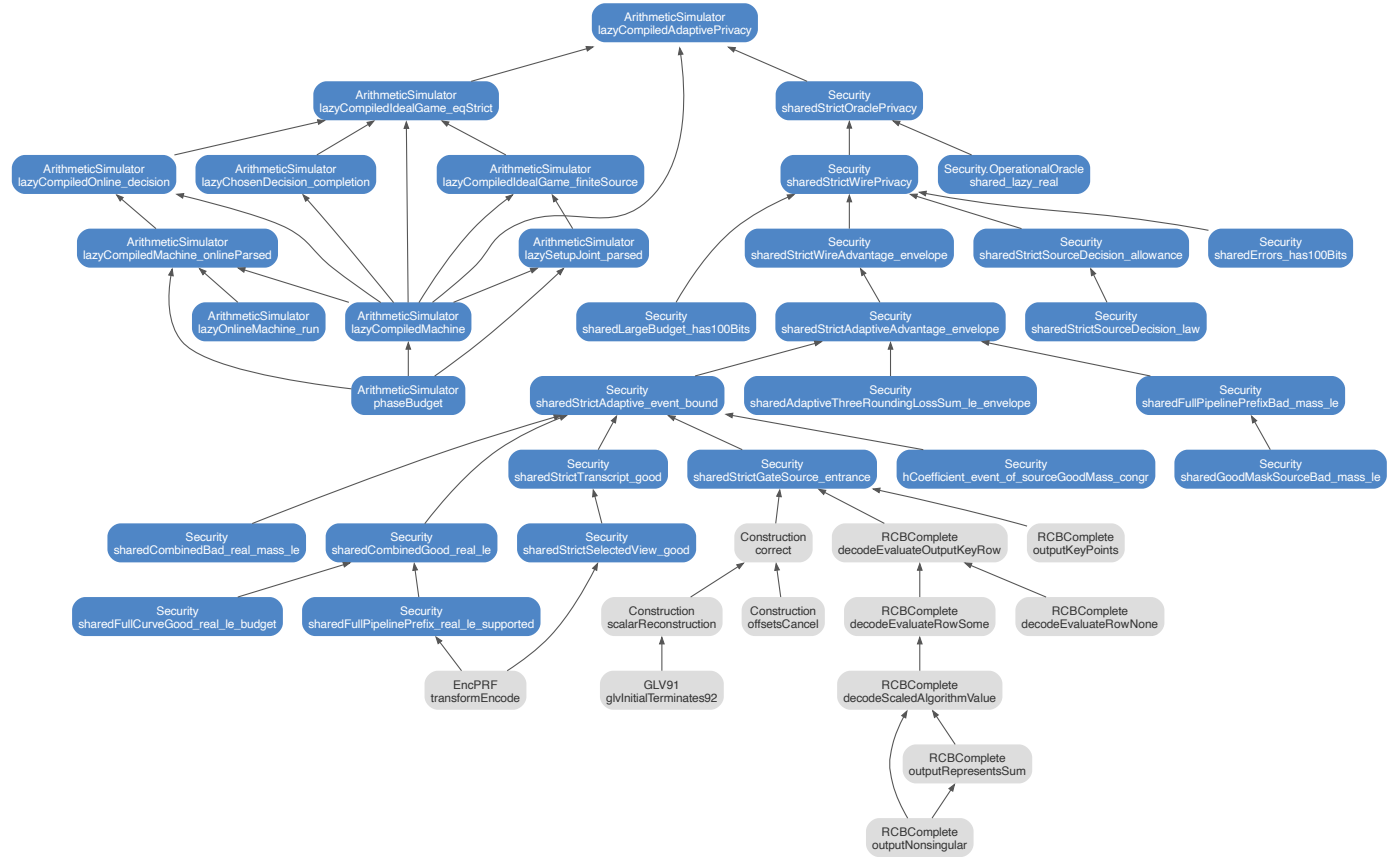


Figure 3.1: Dependencies of the adaptive privacy proof.

3.1 Statements from the paper

Definition 35 (Adaptive privacy; Definition 3 of the BaBe paper [1]). A garbling scheme is ϵ -adaptively private if there is a simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ such that for every adversary $\mathcal{A}$, every circuit C , and every auxiliary input aux :

$$\left| \Pr[\mathcal{A}(\text{ct}_{\text{GC}}, L_x, \text{aux}) = 1 : (\text{ct}_{\text{GC}}, \text{ek}) \leftarrow \text{Garble}(C), x \leftarrow \mathcal{A}(\text{ct}_{\text{GC}}, \text{aux}), L_x \leftarrow \text{Encode}(\text{ek}, x)] \right. \\ \left. - \Pr[\mathcal{A}(\text{ct}_{\text{GC}}, L_x, \text{aux}) = 1 : (\text{ct}_{\text{GC}}, \text{st}) \leftarrow \text{Sim}_1(\text{topo}(C)), x \leftarrow \mathcal{A}(\text{ct}_{\text{GC}}, \text{aux}), L_x \leftarrow \text{Sim}_2(\text{st}, C(x))] \right| \leq \epsilon.$$

The adversary chooses the input x after it sees the garbled circuit. The Lean statement bounds the advantage by the work of the adversary: $T/\delta \geq 2^{100}$ with $T = q + 1$. Formalized by theorems 45 and 46.

Theorem 36 (Theorem 7 of the BaBe paper [1]). Suppose GC_3 is ϵ_3 -adaptively private and GC_5 is ϵ_5 -adaptively private. Suppose the PRF CTPRF used in GC_3 and GC_5 is modeled as a programmable random permutation and (Enc, Dec) is ϵ_2 -CPA secure. Then the garbling scheme GC of Fig. 21 is $(\epsilon_2 + \epsilon_3 + \epsilon_5)$ -adaptively private. The proof is a sequence of hybrids; Hyb_0 is the real experiment. Formalized by theorem 47 for Hyb_0 .

Theorem 37 (Theorem 8 of the BaBe paper [1]). Suppose the PRF CTPRF used in GC_{23} and GC_{45} is modeled as a programmable random permutation. Suppose (Enc, Dec) is ϵ_2 -CPA secure and is implemented with EncPRF as in theorem 7, where each permutation $P_{w,i}$ is a programmable random permutation and SHA256 is a random oracle. Then the optimized garbling scheme GC^{opt} of Fig. 29 satisfies $(\epsilon_2 + \text{negl}(\lambda))$ -adaptive privacy. Formalized by theorems 45 and 46.

Lemma 38 (Key insight; Lemma 12 of the BaBe paper [1]). Fix a deterministic adversary. For every attainable transcript τ and every world $b \in \{\text{real}, \text{ideal}\}$: $\Pr_{\omega \leftarrow \Omega_b}[Z_b(\omega) = \tau] = \Pr_{\omega \leftarrow \Omega_b}[\omega \in \text{comp}_b(\tau)]$, where Z_b is the transcript of the interaction with the oracle ω and $\text{comp}_b(\tau)$ is the set of oracles compatible with τ .

Lemma 39 (H-coefficient technique; Lemma 13 of the BaBe paper [1]). Let $\mathcal{T}_{\text{bad}}$ be a set of bad transcripts and $\mathcal{T}_{\text{good}}$ its complement in the attainable transcripts. Suppose there are $\epsilon_1, \epsilon_2 \geq 0$ such that $\sum_{\tau \in \mathcal{T}_{\text{bad}}} \Pr_{\omega \leftarrow \Omega_{\text{ideal}}}[\omega \in \text{comp}_{\text{ideal}}(\tau)] \leq \epsilon_1$ and, for every $\tau \in \mathcal{T}_{\text{good}}$, $1 - \Pr_{\omega \leftarrow \Omega_{\text{real}}}[\omega \in \text{comp}_{\text{real}}(\tau)] / \Pr_{\omega \leftarrow \Omega_{\text{ideal}}}[\omega \in \text{comp}_{\text{ideal}}(\tau)] \leq \epsilon_2$. Then the advantage of the adversary is at most $\epsilon_1 + \epsilon_2$. Formalized by theorems 51 and 52.

Lemma 40 (Adaptive privacy of GC_3 in the pRPM; Lemma 14 of the BaBe paper [1] and the remark after it). Let CTPRF be instantiated from fixed-key AES-128 with one independent permutation per bucket (i, k, W, slot) , each modeled as a programmable random permutation at $\lambda = 128$. Let $n = 254$, let $B = 6858$ be the number of permutations, let $Q_{\text{act}}^{(b)}$ be the number of active gates per bucket, and let q be the total number of permutation queries of the adversary. Then GC_3 (Construction 2) with the simulator of Construction 3 is ϵ_3 -adaptively private in the pRPM with

$$\epsilon_3 \leq \frac{3B(Q_{\text{act}}^{(b)})^2 + 4Q_{\text{act}}^{(b)}q}{2^\lambda}.$$

With the binary digits, $Q_{\text{act}}^{(b)} = n = 254$ and the work-per-advantage level $\min_{\mathcal{A}} \log_2(T_{\mathcal{A}}/\delta_{\mathcal{A}})$ is at least 97.6 bits for every q . With the base- $(2 - \omega)$ digits of the implementation, $Q_{\text{act}}^{(b)} = \ell = 92$ and the level is at least 100 bits: $T/\delta \geq 2^{100.62}$. Formalized by theorems 48 to 50, 62, 65 and 66, with the Lean envelope $\epsilon_3(q) = (251850146 + 833q)/2^{128}$.

Definition 41 (Bad transcripts; Definition 20 of the BaBe paper [1]). For a transcript $\tau = (K_1, \text{ct}_{\text{GC},3}, K_2, (\mathbf{b}, L_3), K_3)$, let $a_{i,j,k,W}^* = L_{3,\delta(k)}^{(i)} \oplus \text{enc}(j, k, W)$ be the active permutation input of gate (i, j, k, W) , let $v_{i,j,k,W}^*$ be its required output, and let $w_{i,j,k,W}$ be its inactive output offset. A transcript is bad if one of the following events occurs.

bad1 Two distinct gates have the same active input a^* .

bad2 Two distinct gates have the same required output v^* .

bad3 A permutation query $(\alpha, \beta) \in K_1 \cup K_2$ made before the labels are released hits an active input, $\alpha = a^*$, or a required output, $\beta = v^*$.

bad4 Two distinct gates that share a wire label have the same inactive offset w .

Formalized by theorems 57 and 58: the strict simulator aborts exactly on a bad sample.

Claim 42 (Bad-transcript mass; Claim 11 of the BaBe paper [1]). $\varepsilon_1 = \sum_{\tau \in \mathcal{T}_{\text{bad}}} \Pr_{\omega \leftarrow \Omega_{\text{ideal}}}[\omega \in \text{comp}_{\text{ideal}}(\tau)] \leq \frac{\frac{3}{2}Q_{\text{act}}^2 + 2qQ_{\text{act}}}{2^\lambda}$, where *bad1*, *bad2*, and *bad4* each contribute at most $Q_{\text{act}}^2/2^{\lambda+1}$ and *bad3* contributes at most $2qQ_{\text{act}}/2^\lambda$. Formalized by theorems 53, 60 and 61.

Claim 43 (Good-transcript ratio; Claim 12 of the BaBe paper [1]). For every good transcript τ : $\Pr_{\omega \leftarrow \Omega_{\text{real}}}[\omega \in \text{comp}_{\text{real}}(\tau)] / \Pr_{\omega \leftarrow \Omega_{\text{ideal}}}[\omega \in \text{comp}_{\text{ideal}}(\tau)] \geq 1 - \delta_2$ with $\delta_2 = (3Q_{\text{act}}^2 + 2qQ_{\text{act}})/2^\lambda$, so $\varepsilon_2 = \delta_2$. Formalized by theorems 54 to 56.

Construction 44 (Simulator for GC_3 ; Construction 3 of the BaBe paper [1]). $\text{Sim}_1(\text{topo}(C_3))$ samples every ciphertext $\text{ct}_{3,b}^{(i,j,k,W)} \leftarrow \{0,1\}^\lambda$ uniformly and outputs $\text{ct}_{\text{GC},3}$ as its state. $\text{Sim}_2(\text{st}, \mathbf{b}, \{c_{k,i}'^{(j,W)}\})$ samples the active labels $L_{3,x}^{(i)}, L_{3,y}^{(i)} \leftarrow \{0,1\}^\lambda$, and for every gate programs the permutation at $L_{3,\delta(k)}^{(i)} \oplus \text{enc}(j, k, W)$ so that CTPRF decrypts the active ciphertext to the correct output $c_{k,i}'^{(j,W)}$; it outputs L_3 . The Lean simulator compiles $(\text{Sim}_1, \text{Sim}_2)$ to a bounded machine with at most 2^{60} instructions, and bounds every rejection-sampling draw to 256 attempts. Formalized by theorems 63, 64, 67 and 74.

3.2 Lean statements

3.2.1 Closed simulator

Theorem 45 (BABE Theorem 8). *Lean: Kriterion.ArgoMAC.ArithmeticSimulator.lazyCompiledAdaptivePrivacy* Adaptive privacy of the optimized garbling scheme (Theorem 8) at the concrete level of Lemma 14. The compiled simulator $(\text{Sim}_1, \text{Sim}_2)$ proves the public privacy obligation for the program circuit, the Lamport wire encoding, the ciphertext size 9806076 bytes, and the uniform private tape: every adversary with work T has advantage δ with $T/\delta \geq 2^{100}$. The proof uses no hardness assumption beyond the programmable random permutation model.

Proof. Apply theorem 46 to the compiled machine. Theorem 67 shows that the machine implements the strict simulator. $\square$

Theorem 46 (BABE Theorem 8). *Lean: Kriterion.ArgoMAC.Security.sharedStrictOraclePrivacy* A closed implementation of the strict simulator meets the privacy rule of Definition 3. If a bounded machine implements the ideal world of $(\text{Sim}_1, \text{Sim}_2)$ with 256 sampling attempts per draw, then the program circuit satisfies oracle adaptive privacy with $T/\delta \geq 2^{100}$.

Proof. Use the given machine as the simulator. Rewrite the real world with theorem 47 and the ideal world with the implementation premise. Apply theorem 48 to the machine adversary. The machine step count T dominates the query budget. $\square$

Theorem 47 (BABE Theorem 7). *Lean: Kriterion.ArgoMAC.Security.OperationalOracle.shared_lazy_real*

The real world $\mathcal{O}_{\text{real}}$ splits into private coins and the public permutation (hybrid Hyb_0 in the proof of Theorem 7). The lazy real game, which samples the public permutation on demand, equals the real game of the wire circuit with a uniformly sampled tape.

Proof. Rewrite the lazy real game with the uniform tape. Split the uniform randomness into the private coins and the public permutation. Both games run the adversary under a public handler. The permutation state does not change during the choose phase, so both games evaluate the same experiment. $\square$

Theorem 48 (BABE Lemma 14). *Lean: Kriterion.ArgoMAC.Security.sharedStrictWirePrivacy*

Concrete adaptive privacy (Lemma 14). For every adversary with query budget q , the advantage δ between the real world of the wire circuit and the strict simulator with 256 attempts per draw satisfies $(q + 1)/\delta \geq 2^{100}$.

Proof. Let q be the total query budget. Transfer the advantage through the exact strict simulator. If $q < 2^{101}$, apply theorem 65 with theorem 49 and theorem 63. Otherwise apply theorem 66. $\square$

3.2.2 Privacy bound

Theorem 49 (BABE Lemma 14). *Lean: Kriterion.ArgoMAC.Security.sharedStrictWireAdvantage_envelope*

For $q \leq 2^{101}$, the advantage between the real world of the wire circuit and the exact strict simulator is at most $\varepsilon_3(q) = (251850146 + 833q)/2^{128}$, the Lean counterpart of the bucketed bound of Lemma 14.

Proof. Rewrite the exact strict simulator. Apply theorem 50 to the wire adversary. The wire circuit is the internal circuit with Lamport-mapped labels, so the two real worlds agree. $\square$

Theorem 50 (BABE Lemma 14). *Lean: Kriterion.ArgoMAC.Security.sharedStrictAdaptiveAdvantage_envelope*

For $q \leq 2^{101}$, the advantage between the real world of the internal circuit and the ideal world of the strict simulator $(\text{Sim}_1, \text{Sim}_2)$ is at most $\varepsilon_3(q)$.

Proof. Apply theorem 51 to the event $\{\text{true}\}$ with the bad-transcript mass of theorem 60. The real decision is the projection of the real transcript. Theorem 62 collects $\varepsilon_1, \varepsilon_2$, and the sampling losses into $\varepsilon_3(q)$. $\square$

Theorem 51 (BABE Lemma 13). *Lean: Kriterion.ArgoMAC.Security.sharedStrictAdaptive_event_bound*

H-coefficient bound for the strict simulator (Lemma 13). For every event on the decision bit, the real-world transcript and the ideal-world transcript differ by at most the bad-transcript mass ε_1 , which is the given offline bound plus $(60199524 + 372q)/2^{128}$ plus $(q+1)/p$, plus two hash-rounding losses plus 2^{-240} .

Proof. Bound the bad-transcript mass with theorem 53. Apply theorem 52 to the real transcript and the ideal sample space. The ratio premise $1 - \varepsilon_2$ is theorem 54. The agreement premise on good transcripts is theorem 57. The strict decision is the projection of the strict transcript. Add the sampling loss of theorem 59 by the triangle inequality. $\square$

Theorem 52 (BABE Lemma 13). *Lean: Kriterion.ArgoMAC.Security.hCoefficient_event_of_sourceGoodMass_congr*

The H-coefficient technique (Lemma 13) in event form. Let real be a distribution on transcripts, let the ideal sample space have a bad set $\mathcal{T}_{\text{bad}}$ of mass at most ε_1 , and let a kernel map samples to transcripts. If $\Pr_{\text{real}}[\tau] \geq (1 - \varepsilon_2) \cdot \Pr[\text{good sample} \mapsto \tau]$ for every τ , and a replacement kernel agrees with the kernel on good samples, then every event differs between real and the replacement by at most $\varepsilon_1 + \varepsilon_2$.

Proof. Apply the H-coefficient event bound to the replacement kernel. The good mass of the replacement equals the good mass of the kernel, because both agree on every supported sample outside $\mathcal{T}_{\text{bad}}$. The ratio premise transfers. $\square$

Theorem 53 (BABE Claim 11). *Lean: Kriterion.ArgoMAC.Security.sharedCombinedBad_real_mass_le*
Bad-transcript mass ε_1 (Claim 11). The mass of the bad event of the ideal sample space is at most the given offline bound (collisions among the programmed points, events bad1 , bad2 , bad4) plus the hash-rounding loss plus $(q + 1)/p$ (the adversary hits a programmed point, event bad3).

Proof. The combined bad mass is at most the offline bad mass plus the mass of a hit on a programmed point. The first term is the offline bound. The second term is the exact bound for a hit. $\square$

Theorem 54 (BABE Claim 12). *Lean: Kriterion.ArgoMAC.Security.sharedCombinedGood_real_le*
Good-transcript ratio (Claim 12). For $q \leq 2^{101}$ and every transcript τ : $(1 - \varepsilon_2) \cdot \Pr_{\text{ideal}}[\text{good sample} \mapsto \tau] \leq \Pr_{\text{real}}[\tau]$ with $\varepsilon_2 = (60199524 + 372q)/2^{128}$, for an on-curve and for an off-curve chosen input.

Proof. Split on whether the chosen input is on the curve. For an on-curve input, the good mass is at most the offline good mass. If that mass is zero the claim is trivial. Otherwise theorem 55 applies with the query length of a supported transcript. For an off-curve input, the good mass is at most the curve good mass, and theorem 56 applies. $\square$

Theorem 55 (BABE Claim 12). *Lean: Kriterion.ArgoMAC.Security.sharedFullPipelinePrefix_real_le_supported*

Good-transcript ratio for an on-curve input. For every transcript τ whose chosen input is on the curve and whose permutation queries number at most $q \leq 2^{101}$: $(1 - (60199016 + 368q)/2^{128}) \cdot \Pr_{\text{ideal}}[\text{good offline sample} \mapsto \tau] \leq \Pr_{\text{real}}[\tau]$.

Proof. If the good mass is zero the claim is trivial. Otherwise expand the good mass as a weighted sum and pick a nonzero term. The term supplies compatible active labels, input bits, and a label vector. The supported-ratio lemma for the on-curve branch gives the bound. $\square$

Theorem 56 (BABE Claim 12). *Lean: Kriterion.ArgoMAC.Security.sharedFullCurveGood_real_le_budget*

Good-transcript ratio for an off-curve input. For every transcript τ whose chosen input is off the curve and $q \leq 2^{101}$: $(1 - \varepsilon_2) \cdot \Pr_{\text{ideal}}[\text{good sample} \mapsto \tau] \leq \Pr_{\text{real}}[\tau]$. Only the curve membership layer C_4, C_5 is compared, because the labels L_3 stay encrypted.

Proof. If the good mass is zero the claim is trivial. Otherwise expand the good mass as a weighted sum and pick a nonzero term. The term bounds the query length by q . The supported-ratio lemma for the off-curve branch gives the bound. $\square$

Theorem 57 (BABE Definition 20). *Lean: Kriterion.ArgoMAC.Security.sharedStrictTranscript_good*
On a good sample, the strict simulator does not abort. For every sample of the ideal space outside $\mathcal{T}_{\text{bad}}$ (Definition 20), the strict transcript equals the sampled transcript.

Proof. Unfold the ideal sample space to find the prior draw of the sample. The offline phase does not abort on a good sample. Without an abort the strict transcript equals the sampled transcript. The abort lemmas reduce to theorem 58. $\square$

Theorem 58 (BABE Definition 20). *Lean: Kriterion.ArgoMAC.Security.sharedStrictSelectedView_good*

The programming of Sim_2 succeeds on a good tag. When the garbled C_2, C_3 tag is good and the permutation state starts without a conflict, programming the active labels of the chosen input (Construction 3, Step 3) does not raise the bad flag, on the on-curve and on the off-curve branch.

Proof. Split on whether the input is on the curve. In both branches, rewrite the program view as the source view. The programming commands of a good tag stay fresh, so the bad flag stays false. $\square$

Theorem 59 (BABE Lemma 14). *Lean: Kriterion.ArgoMAC.Security.sharedStrictGateSource_entrance*
Sampling loss of the real oracle. The real gate source, which samples $\mathbb{F}_p$ elements by reducing 384-bit hash outputs, and the ideal gate source differ on every event by at most the hash-rounding loss $305054 \cdot (2^{384} \bmod p)/2^{384}$ plus 2^{-240} .

Proof. This is the gate source observation bound applied to the strict observer. $\square$

3.2.3 Collision bounds

Theorem 60 (BABE Claim 11). *Lean: Kriterion.ArgoMAC.Security.sharedFullPipelinePrefixBad_mass_le*

Bad-transcript mass of the offline phase (Claim 11, events bad1, bad2, bad4). The probability that the programmed points of Sim_2 collide is at most $188023005.716/2^{128} + 368 q_1/2^{128}$ plus the hash-rounding loss, where q_1 is the number of queries before the chosen input.

Proof. The bad offline event is the image of the retained bad source under the offline law. By theorem 61 the retained bad source has the joint collision bound. Add the hash-rounding loss. $\square$

Theorem 61 (BABE Claim 11). *Lean: Kriterion.ArgoMAC.Security.sharedGoodMaskSourceBad_mass_le*
Joint collision bound (Claim 11). For the independent mask source of the ideal world, the collision flag has mass at most $188023005.716/2^{128} + 368 q_1/2^{128}$.

Proof. Bound the mass by the concrete retained flags. Unfold both sources into a product of uniform draws. For each retained tape, the observer flags have the checked collision mass. $\square$

Theorem 62 (BABE Lemma 14). *Lean: Kriterion.ArgoMAC.Security.sharedAdaptiveThreeRoundingLossSum_le_envelope*

Arithmetic of Lemma 14. The sum $\varepsilon_1 + \varepsilon_2$ plus three hash-rounding losses plus 2^{-240} is at most $\varepsilon_3(q) = (251850146 + 833q)/2^{128}$ when $q_1 \leq q$.

Proof. The field mask loss is at most the block loss, so $(q+1)/p \leq (q+1)/2^{128}$. The term 2^{-240} is at most 2^{-128} . Sum all terms and compare with the constant of ε_3 by linear arithmetic. $\square$

3.2.4 Finite sampling

Theorem 63 (BABE Construction 3). *Lean: Kriterion.ArgoMAC.Security.sharedStrictSourceDecision_allowance*

Bounded rejection sampling. The advantage between the exact strict simulator and the strict simulator with 256 attempts per draw is at most the machine cutoff allowance of q . The simulator of Construction 3 samples with unbounded rejection; the Lean simulator bounds each draw.

Proof. The total law of theorem 64 with 256 attempts bounds the advantage. Unfold the cutoff allowance and compare the coefficients. $\square$

Theorem 64 (BABE Construction 3). *Lean: Kriterion.ArgoMAC.Security.sharedStrictSourceDecision_law*

The strict simulator draws 917653 field elements by rejection sampling. With k attempts per draw, the simulator with bounded draws is a total law of the exact simulator: it agrees with the exact simulator unless some draw fails k times.

Proof. Unfold the strict simulator. The offline sampler Sim_1 has a total law with 917470 draws. The online sampler Sim_2 adds 183 draws. The remaining steps are exact. $\square$

Theorem 65 (BABE Lemma 14). *Lean: Kriterion.ArgoMAC.Security.sharedErrors_has100Bits*
Security level (Lemma 14). If the privacy error is at most $\varepsilon_3(q)$ and the implementation error is at most the cutoff allowance of q , then $(q + 1)/(\text{privacy error} + \text{implementation error}) \geq 2^{100}$.

Proof. Add the two bounds. Multiply by 2^{100} . The envelope ε_3 plus the cutoff allowance has 100 bits of work. $\square$

Theorem 66 (BABE Lemma 14). *Lean: Kriterion.ArgoMAC.Security.sharedLargeBudget_has100Bits*
Large budgets. When $q \geq 2^{101}$, the sum of two advantages, each at most 1, satisfies $(q + 1)/(\delta + \delta') \geq 2^{100}$.

Proof. Each advantage is at most one, so the sum is at most two. Multiply by 2^{100} . The budget plus one is at least 2^{101} , which dominates. $\square$

3.2.5 Simulator execution

Theorem 67 (BABE Construction 3).

Lean: Kriterion.ArgoMAC.ArithmeticSimulator.lazyCompiledIdealGame_eqStrict

The compiled machine implements the strict simulator. The ideal world of the compiled machine equals the ideal world of $(\text{Sim}_1, \text{Sim}_2)$ (Construction 3) with 256 attempts per draw.

Proof. Apply theorem 68. For each setup in the support, apply theorem 70. The online premise is theorem 71 with the offline coin stored in memory. $\square$

Theorem 68 (BABE Construction 3).

Lean: Kriterion.ArgoMAC.ArithmeticSimulator.lazyCompiledIdealGame_finiteSource

If the online phase of the compiled machine equals, for every setup in the support of Sim_1 , the strict frame decision of Sim_2 after the public completion of the permutation, then the ideal world of the machine equals the strict simulator.

Proof. Split the parsed game into the setup and online phases and rewrite the setup with theorem 69. Replace the online decision on the support of the setup by the premise. The result is the strict simulator over the joint setup law. $\square$

Theorem 69 (BABE Construction 3).

Lean: Kriterion.ArgoMAC.ArithmeticSimulator.lazySetupJoint_parsed

The setup phase of the compiled machine is Sim_1 . Parsing the memory after the setup run returns the public table ct_{gc} , the memory, and the empty lazy permutation, distributed as the joint offline source.

Proof. Parse the result of the setup machine run. The setup machine law gives the joint offline source. On the support of the joint source, the public value in memory parses to ct_{gc} . $\square$

Theorem 70 (BABE Construction 3).

Lean: Kriterion.ArgoMAC.ArithmeticSimulator.lazyChosenDecision_completion

The choose phase keeps only the permutation fields that affect the strict decision. If the parsed online phase has the completed strict law for every chosen input, then the chosen decision of the machine equals the strict frame decision of Sim_2 after the public completion.

Proof. Complete the choose phase with the online premise. The chosen decision unfolds to the same bind on the support. The strict frame decision agrees with the completed run because the public run keeps the bad flag false. $\square$

Theorem 71 (BABE Construction 3).

Lean: Kriterion.ArgoMAC.ArithmeticSimulator.lazyCompiledOnline_decision

The online phase of the compiled machine is Sim_2 . When the offline coin is stored in memory and the lazy permutation matches the transcript, the parsed online decision equals the strict decision: sample the online randomness with 256 attempts, program the active labels of the chosen input, abort on a conflict, and otherwise run the adversary's decision on the programmed permutation.

Proof. Rewrite the parsed online phase with theorem 72. The lazy online result decision lemma gives the completed strict law. $\square$

Theorem 72 (BABE Construction 3).

Lean: Kriterion.ArgoMAC.ArithmeticSimulator.lazyCompiledMachine_onlineParsed

The online parser returns the active labels L_3 and the programmed permutation. The parsed online phase of the compiled machine equals the lazy online result mapped to the 508 labels of 128 bits.

Proof. Prepare the input memory with the phase dispatch. Run the compiled machine for the online phase. By theorem 73 the online machine returns the lazy online result. Parse the labels from memory word 3. $\square$

Theorem 73 (BABE Construction 3). *Lean: Kriterion.ArgoMAC.ArithmeticSimulator.lazyOnlineMachine_run*

Every online request fits the fuel 2^{46} . Running the online machine on a memory that holds the chosen input and the optional output point returns the lazy online result at program counter 317804843.

Proof. Bound the prefix cost and the sampling budget. Split on the output. Without an output the null run fits the fuel. With an output the valid run fits the fuel bound. $\square$

Definition 74 (BABE Construction 3).

Lean: Kriterion.ArgoMAC.ArithmeticSimulator.lazyCompiledMachine

The compiled simulator $(\text{Sim}_1, \text{Sim}_2)$ as one bounded machine. The phase simulator combines the offline machine with 256 attempts per draw and the lazy online machine under the shared instruction allowance.

Theorem 75 (BABE Construction 3). *Lean: `_private.Proof.Privacy.Simulator.Arithmetic.CompiledMachine.O.Kriterion.ArgoMAC.ArithmeticSimulator.phaseBudget`*

The size of the phase simulator plus both fuel allowances is at most 2^{60} , when the setup size plus fuel is at most 605084290688, the online code has at most 317804844 instructions, and the online fuel is at most 2^{46} .

Proof. Unfold the phase simulator size and use linear arithmetic. □

Chapter 4

Lamport compatibility

The root is theorem 78. The input labels $\text{Encode}(\text{ek}, \pi)$ are exactly the Lamport signature of the bits of π under the key pairs of ek , as the locking scripts of Eqs. 15–16 require. Figure 4.1 shows the proof tree.

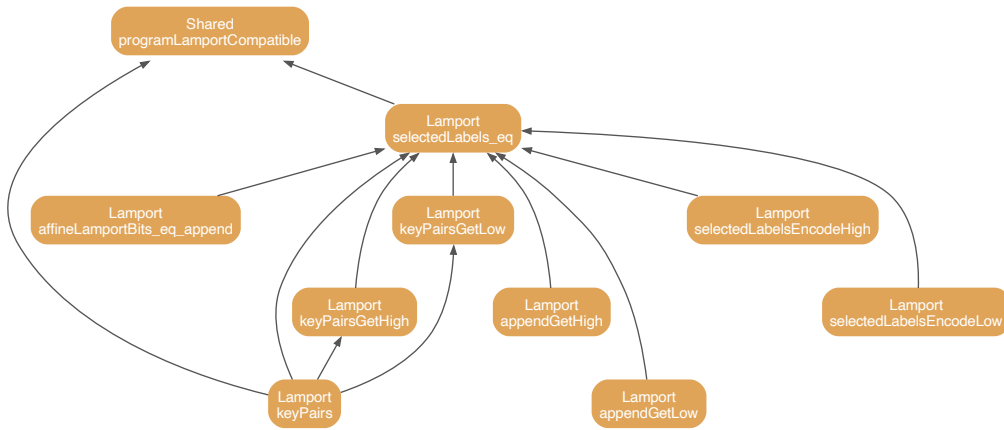


Figure 4.1: Dependencies of the Lamport compatibility proof.

4.1 Statements from the paper

Definition 76 (Bitcoin constraint on the input labels; Section 5.1 of the BaBe paper [1]). The input labels produced by Encode are revealed on-chain. The Prover commits to π by a Lamport signature on the bits $\mathbf{b}$ of its affine coordinates, and the input labels of the garbled circuit are exactly the Lamport signatures of $\mathbf{b}$ under the encoding key ek : for the bit $x_i(\pi)$ the label is $L_{x,i}^{x_i(\pi)}$ and for the bit $y_i(\pi)$ the label is $L_{y,i}^{y_i(\pi)}$. Formalized by theorems 78 and 80.

Definition 77 (Locking scripts; Eqs. 15–16 of the BaBe paper [1]). For a Lamport public key $\text{lpk} = (\text{lpk}_i^0, \text{lpk}_i^1)_{i \leq \ell}$, the script $\text{CheckLampSig}(\text{lpk}) = \bigwedge_{i=1}^{\ell} (\text{HashLock}(\text{lpk}_i^0) \vee \text{HashLock}(\text{lpk}_i^1))$ requires a Lamport signature on some message, and $\text{CheckLampSigsMatch}(\text{lpk}_A, \text{lpk}_B) = \bigwedge_{i=1}^{\ell} [(\text{HashLock}((\text{lpk}_A)_i^0) \wedge \text{HashLock}((\text{lpk}_B)_i^0)) \vee (\text{HashLock}((\text{lpk}_A)_i^1) \wedge \text{HashLock}((\text{lpk}_B)_i^1))]$ requires signatures under both keys on the same message. In ArgoMAC the message is $\mathbf{b}$ with $\ell = 508$ bits, and the key pairs are the labels $(L_{x,i}^0, L_{x,i}^1)$ and $(L_{y,i}^0, L_{y,i}^1)$. Formalized by theorems 11 and 79.

4.2 Lean statements

Definition 78 (BABE Eqs. 15–16).

Lean: `Kriterion.ArgoMAC.Shared.programLamportCompatible`

Lamport compatibility (Eqs. 15–16). The Lamport key pairs $\{(L_{x,i}^0, L_{x,i}^1), (L_{y,i}^0, L_{y,i}^1)\}_{i < 254}$ of ek and the bit decomposition $\mathbf{b}$ of π select exactly the labels $\text{Encode}(\text{ek}, \pi)$, so the input labels are the Lamport signature of π .

Definition 79 (BABE Section 5.1). *Lean:* `Kriterion.ArgoMAC.Lamport.keyPairs`

The Lamport secret key of ek_3 . Slot $i < 254$ holds $(L_{x,i}^0, L_{x,i}^1)$ and slot $i \geq 254$ holds $(L_{y,i-254}^0, L_{y,i-254}^1)$.

Theorem 80 (BABE Eqs. 15–16).

Lean: `Kriterion.ArgoMAC.Lamport.selectedLabels_eq`

$\text{Encode}_3(\text{ek}_3, \mathbf{b}) = \{L_{x,i}^{x_i(\pi)}, L_{y,i}^{y_i(\pi)}\}_{i < 254}$ equals the Lamport signature of the bits $\mathbf{b}$ of π under the key pairs.

Proof. Rewrite $\mathbf{b}$ as an append with theorem 81. Compare index by index. For $i < 254$ use theorem 82, theorem 83, and theorem 84. For $i \geq 254$ use theorem 85, theorem 86, and theorem 87. $\square$

Theorem 81 (BABE Section 5.1). *Lean:* `Kriterion.ArgoMAC.Lamport.affineLamportBits_eq_append`
The bit decomposition $\mathbf{b}$ of π is the 254 bits of $y(\pi)$ appended to the 254 bits of $x(\pi)$.

Proof. Compare the natural values. The appended value is $y(\pi)$ shifted by 254 bits or $x(\pi)$. Both coordinates are below 2^{254} , so no reduction occurs. $\square$

Theorem 82 (BABE Section 5.1). *Lean:* `_private.Proof.LamportCompatibility.Labels.O.Kriterion.ArgoMAC.Lamport.appendGetLow`

For $i < 254$, bit i of the appended vector is bit i of the low vector.

Proof. The bit of an appended vector below the low width reads the low vector. $\square$

Theorem 83 (BABE Section 5.1). *Lean:* `_private.Proof.LamportCompatibility.Labels.O.Kriterion.ArgoMAC.Lamport.selectedLabelsEncodeLow`

For $i < 254$, label i of $\text{Encode}_3(\text{ek}_3, \mathbf{b})$ is $L_{x,i}^{x_i(\pi)}$.

Proof. Unfold the selected labels and the encoding. The low index selects the x coordinate encoding. $\square$

Theorem 84 (BABE Section 5.1).

Lean: `_private.Proof.LamportCompatibility.Labels.O.Kriterion.ArgoMAC.Lamport.keyPairsGetLow`

For $i < 254$, key pair i is $(L_{x,i}^0, L_{x,i}^1)$.

Proof. Unfold the key pairs. The low index selects the x item. $\square$

Theorem 85 (BABE Section 5.1). *Lean:* `_private.Proof.LamportCompatibility.Labels.O.Kriterion.`
ArgoMAC.Lamport.appendGetHigh

For $i \geq 254$, bit i of the appended vector is bit $i - 254$ of the high vector.

Proof. The bit of an appended vector at or above the low width reads the high vector at $i - 254$. $\square$

Theorem 86 (BABE Section 5.1). *Lean:* `_private.Proof.LamportCompatibility.Labels.O.Kriterion.`
ArgoMAC.Lamport.selectedLabelsEncodeHigh

For $i \geq 254$, label i of $\text{Encode}_3(\text{ek}_3, \mathbf{b})$ is $L_{y,i-254}^{y_{i-254}(\pi)}$.

Proof. Unfold the selected labels and the encoding. The high index selects the y coordinate encoding at $i - 254$. $\square$

Theorem 87 (BABE Section 5.1).

Lean: `_private.Proof.LamportCompatibility.Labels.O.Kriterion.` *ArgoMAC.Lamport.keyPairsGetHigh*

For $i \geq 254$, key pair i is $(L_{y,i-254}^0, L_{y,i-254}^1)$.

Proof. Unfold the key pairs. The high index selects the y item at $i - 254$. $\square$

Bibliography

- [1] Sanjam Garg, Dimitris Kolonelos, Mikhail Sergeevitch, Srivatsan Sridhar, and David Tse. BABE: Verifying proofs on Bitcoin made 10,000x cheaper. LaTeX sources, version v2.0, commit e2dcf4d, <https://github.com/babylonlabs-io/BaBe.latex>, 2026.